

CSE 310 Online (B1): Oct, Hex, and Lua

Time: 40 minutes Full Marks: 10

You will extend the lexical analyzer developed for the offline lexical-analysis assignment. Add the following two features to your existing .1 file. All previously specified tokenization, symbol-table, line-counting, and error-reporting rules will remain unchanged.

For the samples below, only the additional entries caused by the new features are shown. Existing tokens, symbol-table printouts, and ordinary log entries are omitted.

1. Octal and Hexadecimal Integer Constants

Recognize octal and hexadecimal integer constants in addition to the existing decimal integer constants.

- An **octal constant** starts with 0, contains at least one additional digit, and all following digits must be in 0-7. Examples: 00, 017, 0755.
- A single 0 will continue to be recognized as the existing token `CONST_INT`.
- A **hexadecimal constant** starts with 0x or 0X and must contain at least one hexadecimal digit from 0-9, a-f, or A-F. Examples: 0x2A, 0Xff, 0x10B7.

For valid constants, print the following tokens.

Input form	Token format
0755	<CONST_OCT, 0755>
0x2A	<CONST_HEX, 0x2A>

The log file must follow the existing format. For example:

```
Line no 3: Token <CONST_OCT> Lexeme 0755 found
Line no 4: Token <CONST_HEX> Lexeme 0x2A found
```

Required error checking

- If a number starts with 0, has at least one additional decimal digit, and contains 8 or 9, report:

```
Line no <n>: Invalid octal constant <lexeme>
```

Example invalid lexemes: 078, 0197, 0089.

- If 0x or 0X is not followed by a hexadecimal digit, or if an alphanumeric continuation contains a non-hexadecimal character, report:

```
Line no <n>: Invalid hexadecimal constant <lexeme>
```

Example invalid lexemes: 0x, 0XG1, 0x2AZ.

- An invalid numeric lexeme must be consumed and reported as **one error**; it must not be split into several valid tokens. No token will be written for an invalid constant. Octal and hexadecimal constants, whether valid or invalid, are not inserted into the scope table.

2. Lua-Style Comments

In addition to the existing C/C++ comments, recognize the following Lua-style comment:

- **Multiline Lua comment:** starts with `--[[` and ends at the first following `]]`. It may span multiple lines.

Lua comments will generate **no token** in the token file and will not be inserted into the symbol table. Add only the following messages to the log file:

```
Line no <start>: Lua multiline comment ending at line <end> found
```

Comment delimiters appearing inside string literals or character constants must be treated as ordinary characters.

Required error checking

If the end of file is reached before a Lua multiline comment is closed, report exactly one error using its starting line:

```
Line no <start>: Unfinished Lua multiline comment
```

The unfinished comment will generate no token. Count this error in the existing total error count.

Sample I/O

Sample 1: Valid numbers and comments

Input

```
int permission = 0755;
int mask = 0x2Af;
--[ [ Lua comment
spanning two lines ]]
```

Additional token-file content

```
<CONST_OCT, 0755> <CONST_HEX, 0x2Af>
```

Additional log-file content

```
Line no 1: Token <CONST_OCT> Lexeme 0755 found
Line no 2: Token <CONST_HEX> Lexeme 0x2Af found
Line no 3: Lua multiline comment ending at line 4 found
```

Sample 2: Invalid octal and hexadecimal constants

Input

```
int a = 0789;
int b = 0x2G7;
int c = 0X;
```

Additional token-file content

```
[no additional token is produced]
```

Additional log-file content

```
Line no 1: Invalid octal constant 0789
Line no 2: Invalid hexadecimal constant 0x2G7
Line no 3: Invalid hexadecimal constant 0X
```

Sample 3: Valid constants around ordinary C tokens

Input

```
count--;
int mode = 017;
```

```
int color = 0XABC9;
```

Additional token-file content

```
<CONST_OCT, 017> <CONST_HEX, 0XABC9>
```

Additional log-file content

```
Line no 2: Token <CONST_OCT> Lexeme 017 found  
Line no 3: Token <CONST_HEX> Lexeme 0XABC9 found
```

The - in count-; must continue to produce the existing INCOP token. Since only newly added output is shown, that token is omitted above.

Sample 4: Unfinished Lua multiline comment

Input

```
int x = 0x1F;  
--[ this comment  
never ends
```

Additional token-file content

```
<CONST_HEX, 0x1F>
```

Additional log-file content

```
Line no 1: Token <CONST_HEX> Lexeme 0x1F found  
Line no 2: Unfinished Lua multiline comment
```

Marks Distribution

- Octal and hexadecimal constants, including correct token generation: **3 marks**
- Invalid octal/hexadecimal pattern detection without token splitting: **3 marks**
- Lua multiline comment recognition without changing the existing decrement operator: **2 marks**
- Unfinished multiline comment detection and correct line counting: **2 marks**

Submission: Submit only your 2205ABC.1 file. Do not submit lex.yy.c, executables, token files, or log files.